

SUMATIVA #8 · TA — Proyecto · Segundo Corte · Semana 9

PROYECTO FIN DE CURSO — ENTREGA 1B

Modulo de Autenticacion JWT + Acceso a Datos con Spring Data JPA

Stack: Java 21 / Spring Boot 3.x · Angular 17+ · PostgreSQL 16 · Redis · Docker Compose

Sumativa #8

Grupal (mismo equipo PFC)

PDF al LMS (SGA)

Semana 9

Título del PFC	[Nombre completo del sistema]
Sumativa	#8 del Segundo Corte
Semana de entrega	Semana 9 — Del 05 al 11 de Enero del 2026
Modalidad	Grupal — mismo equipo formado en el semestre 2025 SPA
Entregable	Informe tecnico en PDF + repositorio Git actualizado
Stack tecnologico	Java 21 LTS + Spring Boot 3.x + Spring Security 6 + JWT (jjwt 0.12) + Spring Data JPA + Hibernate + PostgreSQL 16 + Flyway + Angular 17+ + Redis 7 + Docker Compose
Continuacion de	Entrega 1A (Semana 5): planificacion, diseno y arquitectura

Que construye el equipo en esta entrega

La Entrega 1B es el primer modulo de codigo funcional del PFC. A partir del diseno documentado en la Entrega 1A, el equipo implementa dos componentes centrales de cualquier aplicacion web empresarial:

1. **Modulo de autenticacion stateless:** registro de usuarios, inicio de sesion y emision de tokens JWT con Spring Security 6. El frontend Angular consume estos tokens para acceder a rutas protegidas.
2. **Modulo de acceso a datos:** CRUD completo de al menos una entidad principal del sistema usando Spring Data JPA con Hibernate sobre PostgreSQL 16. Las migraciones de esquema se gestionan con Flyway para garantizar reproducibilidad.

El resultado es una aplicacion ejecutable con `docker compose up` que el docente puede verificar en tiempo real.

Stack tecnologico obligatorio y justificacion

1.1 Tecnologías del backend

Componente	Version minima	Justificacion tecnica
Java	21 LTS	Version de soporte extendido; introduce Virtual Threads (Project Loom) que mejoran el throughput de I/O sin cambiar el codigo de negocio
Spring Boot	3.2.x	Autoconfiguracion, Tomcat/Netty embebido, starters para JPA, Security y Actuator; elimina la configuracion XML
Spring Security	6.x (incluido)	Cadena de filtros configurable; soporte nativo para JWT como Bearer token; integrado con Spring Boot 3
jjwt	0.12.x	Biblioteca de referencia para firmar y verificar tokens JWT con HS256/RS256
Spring Data JPA	3.x (incluido)	Repositorios genericos; elimina el CRUD manual; soporta paginacion, ordenacion y consultas derivadas del nombre del metodo
Hibernate	6.x (incluido)	ORM de referencia para Jakarta EE; genera DDL, gestiona el ciclo de vida de entidades y el cache de primer nivel
Flyway	9.x	Migraciones de BD versionadas en SQL; garantiza que todos los entornos (dev, test, prod) tienen el mismo esquema
Springdoc OpenAPI	2.x	Genera la documentacion Swagger UI en <code>/api/docs</code> a partir de las anotaciones de los controladores
Lombok	1.18.x	Reduce el codigo repetitivo: <code>@Data</code> , <code>@Builder</code> , <code>@RequiredArgsConstructor</code>
Bean Validation	3.x (incluido)	<code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> en los DTOs; activa con <code>@Valid</code> en el controlador

1.2 Tecnologías de infraestructura y frontend

Componente	Version minima	Justificacion tecnica
PostgreSQL	16	Motor relacional con JSONB, full-text search, TIMESTAMPTZ y cumplimiento estricto del estandar SQL
Redis	7.x	Cache en memoria para tokens de revocacion (blacklist JWT), rate limiting y sesiones de larga duracion
Docker Compose	2.x	Orquesta los tres servicios (backend , postgres , redis) en un solo comando; garantiza reproducibilidad
Angular	17+	SPA con Angular Router, HTTP Client con interceptores para adjuntar el JWT, Angular Material para la UI
Nginx	1.25	Sirve el bundle compilado de Angular y actua como reverse proxy al backend en produccion
JUnit 5 + Mockito	5.x / 5.x	Pruebas unitarias de servicios y de integracion de endpoints REST con <code>@SpringBootTest</code> y <code>MockMvc</code>
GitHub Actions	—	Pipeline CI que compila, ejecuta pruebas y construye la imagen Docker en cada push a develop

Por que Redis en esta entrega

JWT es stateless: el servidor no guarda el token. Si un usuario cierra sesion o el token es robado, el servidor no tiene forma de invalidarlo sin una lista de revocacion. Redis almacena los JTI (JWT ID) revocados con TTL igual a la expiracion del token. En cada solicitud, el filtro JWT consulta Redis antes de autorizar. Esta es una decision de seguridad documentada en el ADR-003 del repositorio.

Estructura del informe tecnico

El informe de la Entrega 1B debe presentarse en las siguientes secciones en el orden indicado. Extension recomendada: 20 a 35 paginas sin contar anexos.

2.1 Portada

Misma estructura que la Entrega 1A: universidad, facultad, carrera, asignatura, titulo del PFC, numero de entrega (1B), integrantes con cedula, docente, fecha, URL del repositorio y rama Git entregada (`entrega-1b`).

2.2 Resumen ejecutivo (Abstract)

Parrafo de 200 a 250 palabras que responde: *que sistema se construye, que problema resuelve, que tecnologias se usaron, que resultados concretos se obtuvieron en esta entrega y cuales son las metricas mas relevantes* (tiempo de emision del token, tiempo de respuesta del endpoint de login, cobertura de pruebas). Redactar en tercera persona, tiempo pasado, sin listas.

Por que este resumen importa

El resumen ejecutivo es el primer texto que el evaluador lee. Un resumen bien escrito comunica que el equipo comprende el sistema completo, no solo el codigo. Debe poder leerse de forma independiente del resto del informe.

2.3 Introduccion

1. **Contexto y motivacion:** describir el problema del mundo real que el sistema resuelve, con datos cuantitativos si los hay (cuantos usuarios afecta, cuanto tiempo o dinero se pierde, estadisticas del sector en Ecuador).
2. **Objetivos de la entrega:** listar los 3 o 4 objetivos especificos de la Entrega 1B en forma de verbos de accion (implementar, configurar, demostrar, validar).
3. **Alcance:** que modulos se implementan en esta entrega y cuales quedan para entregas futuras.
4. **Estructura del documento:** breve descripcion de cada seccion del informe.

2.4 Marco teorico

Esta seccion sustenta las decisiones tecnicas con referencias academicas. Incluir obligatoriamente:

- **Autenticacion stateless vs. stateful:** diferencias conceptuales, ventajas del enfoque JWT en arquitecturas SPA/API, referencias a RFC 7519 (JWT), RFC 6749 (OAuth 2.0) y al menos un articulo indexado sobre seguridad en aplicaciones web.
- **Spring Security 6 y el modelo de filtros:** descripcion de la cadena `SecurityFilterChain`, el `AuthenticationManager` y el mecanismo de `UserDetailsService`.
- **ORM y mapeo objeto-relacional:** justificacion del uso de Hibernate/JPA frente a JDBC puro; citar a Walls (2022) y a la documentacion oficial de Hibernate.
- **Migraciones de base de datos con Flyway:** importancia del versionado del esquema en equipos; diferencia entre `V1__` (migracion) y `R__` (repetible).
- **OWASP y seguridad JWT:** referenciar OWASP Top 10 (2021) A07 (Fallas de autentificacion) y las recomendaciones de almacenamiento seguro del token en el cliente Angular.

Minimo **8 referencias** en norma APA 7ma edicion. Al menos 3 deben ser articulos de revistas o conferencias indexadas (IEEE, ACM, Scopus).

2.5 Diseno del sistema

2.5.1 Actualizacion del modelo C4

Incluir el diagrama C4 Nivel 2 (Contenedores) actualizado respecto a la Entrega 1A. Debe mostrar los nuevos contenedores: backend Spring Boot, frontend Angular, PostgreSQL, Redis. Para cada contenedor indicar tecnologia, protocolo de comunicacion y puerto.

2.5.2 Diagrama de clases UML refactorizado

El **diagrama de clases** es el artefacto principal del modelo de diseno en esta entrega. Representa la estructura completa del backend Spring Boot: clases Java reales con sus atributos tipados, metodos publicos relevantes, relaciones de asociacion, herencia e implementacion de interfaces. Debe generarse desde el IDE (IntelliJ IDEA: *Diagrams* > *Show Diagram*) o con una herramienta como PlantUML o Lucidchart, exportado en PNG de alta resolucion.

Que debe mostrar el diagrama de clases de la Entrega 1B

- **Paquetes** como agrupadores: `entity`, `dto`, `service`, `repository`, `controller`, `security`, `config`
- **Clases de entidad** (anotadas con `@Entity`): atributos con tipo Java exacto (`Long`, `String`, `Instant`, `boolean`, `enum`), anotaciones JPA relevantes (`@Id`, `@Column`, `@ManyToOne`, etc.) y visibilidad (`-` privado, `+` publico)
- **Interfaces de repositorio:** `UsuarioRepository` extends `JpaRepository<Usuario, Long>` con los metodos custom declarados (p.ej. `findByEmail`)

- **Clases de servicio:** dependencias inyectadas como campos **final** (constructor injection); metodos del contrato de negocio con sus tipos de retorno
- **DTOs:** clases record o `@Data` que separan la capa HTTP de las entidades JPA; indicar que **no** tienen anotaciones `@Entity`
- **Clases de seguridad:** `JwtService`, `JwtAuthFilter`, `SecurityConfig`, `UserDetailsServiceImpl` con sus dependencias
- **Relaciones:** asociacion ($\rightarrow$), dependencia ($--\rightarrow$), implementacion (`implements`), herencia (`extends`)

Tabla de correspondencia Java $\leftrightarrow$ PostgreSQL para cada entidad implementada:

Atributo Java	Tipo Java	Tipo PostgreSQL	Restricciones y justificacion
id	Long	BIGSERIAL	PK; secuencia PostgreSQL; mejor rendimiento en joins que UUID
email	String	VARCHAR(255)	UNIQUE NOT NULL; indice CREATE UNIQUE INDEX
passwordHash	String	VARCHAR(255)	BCrypt costo 12; nunca serializado (<code>@JsonIgnore</code>)
rol	enum Rol	VARCHAR(20)	CHECK: ROLE_USER, ROLE_ADMIN; no ENUM nativo PostgreSQL
creadoEn	Instant	TIMESTAMPTZ	UTC; gestionado por <code>@CreationTimestamp</code>
activo	boolean	BOOLEAN	Soft delete; DEFAULT TRUE

Repetir para cada entidad del sistema; la tabla completa va en el diccionario de datos (seccion complementaria)

Secciones complementarias al diagrama de clases

Las siguientes secciones **complementan** el diagrama de clases con la perspectiva de la base de datos. No reemplazan al diagrama de clases; lo enriquecen desde la capa de persistencia:

- **Diagrama Entidad-Relacion (DER):** perspectiva de la BD relacional; muestra tablas, PKs, FKs y cardinalidades. Util para entender las migraciones Flyway. Incluir como figura PNG generada desde pgAdmin 4 o DBeaver.
- **Diccionario de datos:** descripcion detallada de cada columna de cada tabla. Ver plantilla en la seccion siguiente.

Diagrama Entidad-Relacion (complementario). Incluir el DER de las entidades implementadas en esta entrega (como minimo la tabla `usuarios` y las del modulo principal). Generado desde pgAdmin 4 (*Tools > ERD Tool*) o DBeaver. Debe mostrar las PKs, FKs, tipos de dato PostgreSQL y cardinalidades.

Diccionario de datos (complementario). Para cada tabla, describir sus columnas con: nombre, tipo PostgreSQL 16, nulo/no nulo, valor por defecto y descripcion del negocio. Presentar como tabla en el informe o como enlace al archivo `docs/diccionario_datos.md` del repositorio.

2.5.3 Diagrama de secuencia: flujo de autenticación JWT

Incluir un diagrama de secuencia UML que muestre el flujo completo:

1. Cliente Angular envía POST /api/auth/login con {email, password}
2. AuthController delega a AuthenticationManager.authenticate()
3. Spring Security consulta UserDetailsService.loadUserByUsername()
4. UserDetailsService consulta UsuarioRepository.findByEmail()
5. PostgreSQL devuelve el usuario; Hibernate lo mapea a la entidad
6. Spring Security verifica el hash BCrypt con PasswordEncoder.matches()
7. JwtService.generateToken() firma el JWT con la clave secreta
8. El servidor devuelve {accessToken, refreshToken, expiresIn}
9. Angular guarda el accessToken en memoria (no en localStorage) y el refreshToken en cookie HttpOnly
10. En cada solicitud posterior, el JwtInterceptor agrega Authorization: Bearer [token]
11. JwtAuthFilter valida el token, consulta Redis (blacklist) y establece el SecurityContext

2.5.4 Script Flyway de migración inicial

```
database/migrations/V1__schema_inicial.sql

1 -- V1__schema_inicial.sql
2 -- Flyway ejecuta este archivo en orden; nunca modificar
3 -- un archivo V ya ejecutado: crear V2__ en su lugar.
4
5 CREATE EXTENSION IF NOT EXISTS "uuid-oss";
6
7 -- Tabla de usuarios
8 CREATE TABLE usuarios (
9   id          BIGSERIAL      PRIMARY KEY,
10  nombre      VARCHAR(100) NOT NULL,
11  email       VARCHAR(255) NOT NULL,
12  password_hash VARCHAR(255) NOT NULL,
13  rol         VARCHAR(20)  NOT NULL DEFAULT 'ROLE_USER',
14  activo      BOOLEAN      NOT NULL DEFAULT TRUE,
15  creado_en   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
16  actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
17 );
18
19 CREATE UNIQUE INDEX idx_usuarios_email ON usuarios(email);
20
21 ALTER TABLE usuarios
22   ADD CONSTRAINT chk_rol
```

```

23 CHECK (rol IN ('ROLE_USER', 'ROLE_ADMIN'));
24
25 -- [Agregar aqui las tablas de las entidades del dominio del PFC]
26 -- Ejemplo: productos, pedidos, categorias, etc.
27 -- Respetar la convencion snake_case para nombres de columnas.
28
29 -- Trigger para actualizar automaticamente actualizado_en
30 CREATE OR REPLACE FUNCTION set_actualizado_en()
31 RETURNS TRIGGER AS $$
32 BEGIN
33     NEW.actualizado_en = NOW();
34     RETURN NEW;
35 END;
36 $$ LANGUAGE plpgsql;
37
38 CREATE TRIGGER trg_usuarios_actualizado_en
39 BEFORE UPDATE ON usuarios
40 FOR EACH ROW EXECUTE FUNCTION set_actualizado_en();

```

2.6 Implementacion

2.6.1 Estructura del repositorio monorepo

Presentar el arbol de directorios real del repositorio al momento de la entrega, con una descripcion de cada carpeta principal:

Ruta	Contenido
backend/src/main/java/	Codigo fuente Java organizado en paquetes por capa
backend/src/main/resources/	application.yml, plantillas Thymeleaf, migraciones Flyway
backend/src/test/	Pruebas unitarias JUnit 5 + pruebas de integracion MockMvc
frontend/src/app/	Modulos Angular: auth/, core/, shared/, features/
frontend/src/environments/	environment.ts (dev) y environment.prod.ts
database/migrations/	Scripts Flyway V1__, V2__, etc.
docker-compose.yml	Servicios: backend, postgres, redis, frontend (Nginx)
.env.example	Variables: DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET, JWT_EXPIRATION, REDIS_URL
docs/adr/ADR-003-jwt-redis.md	Justificacion de usar Redis para blacklist JWT
.github/workflows/ci.yml	Pipeline GitHub Actions: compile, test, docker build

2.6.2 Configuración de Spring Boot

backend/src/main/resources/application.yml

```
1 spring:
2   datasource:
3     url: ${DB_URL:jdbc:postgresql://localhost:5432/pfc_db}
4     username: ${DB_USER:pfc_user}
5     password: ${DB_PASSWORD:changeme}
6     driver-class-name: org.postgresql.Driver
7   jpa:
8     hibernate:
9       ddl-auto: validate           # Flyway gestiona el DDL
10    show-sql: false                # true solo en desarrollo local
11    open-in-view: false            # evitar LazyInitializationException
12    properties:
13      hibernate:
14        format_sql: true
15        default_schema: public
16    flyway:
17      enabled: true
18      locations: classpath:db/migration
19      baseline-on-migrate: true
20    data:
21      redis:
22        host: ${REDIS_HOST:localhost}
23        port: ${REDIS_PORT:6379}
24
25  security:
26    jwt:
27      secret: ${JWT_SECRET}        # min. 256 bits, generado con openssl
28                                     rand -hex 32
29      expiration-ms: 3600000        # 1 hora
30      refresh-expiration-ms: 60480000 # 7 dias
31
32  springdoc:
33    api-docs:
34      path: /api/docs
35    swagger-ui:
36      path: /api/swagger-ui.html
37      operationsSorter: method
38
39  management:
40    endpoints:
41      web:
42        exposure:
43          include: health,info,metrics
```


2.6.3 Implementacion del filtro JWT

Describir el código del `JwtAuthenticationFilter` que extiende `OncePerRequestFilter`. El informe debe incluir el código completo o un enlace al archivo en el repositorio, e indicar:

- Como se extrae el token del encabezado `Authorization: Bearer [token]`
- Como se valida la firma y la expiracion con `JwtService.validateToken()`
- Como se consulta Redis para verificar que el JTI no esta en la blacklist
- Como se establece el `UsernamePasswordAuthenticationToken` en el `SecurityContextHolder`

2.6.4 Endpoints implementados

Listar todos los endpoints disponibles al momento de la entrega. Usar la siguiente tabla:

Metodo	URL	Auth	Descripcion
POST	/api/auth/registro	No	Registrar nuevo usuario; devuelve el usuario creado (sin hash)
POST	/api/auth/login	No	Autenticar; devuelve accessToken y refreshToken
POST	/api/auth/logout	JWT	Agrega el JTI a la blacklist Redis
POST	/api/auth/refresh	RefreshToken	Emite un nuevo accessToken sin re-autenticar
GET	/api/usuarios/me	JWT	Devuelve el perfil del usuario autenticado
GET	/api/[entidad]	JWT	Listar con paginacion: ?page=0&size=10&sort=id,asc
GET	/api/[entidad]/{id}	JWT	Buscar por ID; 404 si no existe
POST	/api/[entidad]	JWT	Crear; valida con @Valid; 201 Created
PUT	/api/[entidad]/{id}	JWT	Actualizar completo; 200 OK
DELETE	/api/[entidad]/{id}	JWT	Soft delete (activo=false); 204 No Content
GET	/api/docs	No	Swagger UI generado por Springdoc OpenAPI

2.6.5 Estructura del JWT emitido

Documentar el payload exacto del token que emite el sistema. Incluir el decodificado en Base64:

Claim	Valor de ejemplo	Descripcion
sub	"42"	ID del usuario (sujeto del token)
email	"juan@uteq.edu.ec"	Email del usuario
rol	"ROLE_USER"	Rol Spring Security
iat	1735689600	Issued at (timestamp Unix)
exp	1735693200	Expiration (iat + 3600 segundos)
jti	"550e8400-e29b..."	JWT ID unico; usado para blacklist en Redis

2.6.6 Implementacion Angular: interceptor JWT

Describir el `JwtInterceptor` de Angular que adjunta el token en cada solicitud HTTP protegida. Indicar:

- Donde se almacena el `accessToken` en el cliente (en memoria, en una variable del servicio `AuthService`, no en `localStorage`)
- Como el interceptor clona la solicitud y agrega el encabezado `Authorization`
- Como el `AuthGuard` protege las rutas de Angular usando `canActivate`
- Como se maneja el error 401 para redirigir al login

2.7 Pruebas

2.7.1 Pruebas unitarias del servicio de autenticacion

Incluir el código de al menos 5 pruebas JUnit 5 que cubran:

#	Caso de prueba	Metodo de prueba	Esperado	Resultado
1	Login con credenciales correctas	<code>loginExitoso()</code>	200 OK + JWT	[PASS/FAIL]
2	Login con contraseña incorrecta	<code>loginClaveIncorrecta()</code>	401 Unauthorized	[PASS/FAIL]
3	Registro con email ya existente	<code>registroEmailDuplicado()</code>	409 Conflict	[PASS/FAIL]
4	Acceso a endpoint protegido sin token	<code>accesoSinToken()</code>	401 Unauthorized	[PASS/FAIL]
5	Acceso a endpoint protegido con token valido	<code>accesoConTokenValido()</code>	200 OK	[PASS/FAIL]
Agregar pruebas adicionales para cada endpoint del CRUD				

Incluir captura de pantalla de la salida de `mvn test` con todos los tests en verde.

2.7.2 Pruebas de integracion con Postman

Incluir capturas de Postman (o coleccion JSON exportada en el repositorio) que demuestren:

1. **Registro** de un nuevo usuario con datos validos
2. **Login** exitoso con el token recibido visible en la respuesta
3. **Acceso protegido** usando el Bearer token en el encabezado
4. **CRUD completo** de la entidad principal: POST, GET lista, GET por ID, PUT, DELETE
5. **Paginacion**: respuesta de GET con parametros `page` y `size`
6. **Swagger UI**: captura de la interfaz con los endpoints documentados

2.7.3 Métricas de rendimiento

Medir y reportar las siguientes metricas con herramientas como JMeter, k6 o simplemente el historial de Postman:

Endpoint	Tiempo promedio	Tiempo P95	Observacion
POST /api/auth/login	[ms]	[ms]	Incluye BCrypt verify (costo 12)
GET /api/[entidad] (50 registros)	[ms]	[ms]	Sin cache
POST /api/[entidad]	[ms]	[ms]	Con validacion y commit
GET /api/[entidad] con Redis cache	[ms]	[ms]	Comparar con sin cache

2.8 Seguridad implementada

Documentar los controles de seguridad activos en esta entrega:

Control	Implementacion	OWASP Top 10	Evidencia
Hash de contraseñas	BCrypt costo 12 via BCryptPasswordEncoder	A02 Fallas criptograficas	Captura BD: hash \$2a\$12\$...
Tokens JWT de corta vida	accessToken: 1h; refreshToken: 7d	A07 Fallas de autentica-cion	Payload decodificado
Blacklist de tokens	JTI revocado en Redis con TTL	A07	Prueba de logout + intento de reuso
Cabeceras de seguridad	X-Content-Type-Options, X-Frame-Options, CSP via Spring Security	A05 Config. incorrecta	curl -I al endpoint
CORS configurado	Origenes permitidos explicitamente en SecurityConfig	A05	Prueba desde dominio no permitido
Sin SQL directo	Spring Data JPA + Flyway; cero concatenacion SQL	A03 Inyeccion	Revision de codigo
Roles de acceso	ROLE_USER y ROLE_ADMIN con @PreAuthorize	A01 Control de acceso	Prueba de acceso denegado

2.9 Docker Compose y despliegue

Incluir el `docker-compose.yml` completo que el docente usara para verificar la aplicacion. Debe contener los servicios `postgres`, `redis`, `backend` y `frontend` con sus variables de entorno, puertos y dependencias de arranque (`depends_on` con `healthcheck`).

Incluir las instrucciones de ejecucion:

Instrucciones de ejecucion — README.md

```
# 1. Clonar el repositorio y cambiar a la rama de entrega
git clone https://github.com/[equipo]/[repo].git
cd [repo]
git checkout entrega-1b

# 2. Copiar variables de entorno
cp .env.example .env
# Editar .env con las credenciales del entorno local

# 3. Levantar todos los servicios
docker compose up --build -d

# 4. Verificar que todos los servicios estan en estado healthy
docker compose ps

# 5. Acceder a la aplicacion
# Frontend: http://localhost:4200
# Swagger UI: http://localhost:8080/api/swagger-ui.html
# Actuator health: http://localhost:8080/actuator/health

# 6. Ejecutar pruebas (sin Docker)
cd backend && ./mvnw test
```

2.10 Historial de Git

Incluir una captura del historial de commits (`git log -oneline -graph -all`) que muestre:

- Commits de **todos** los integrantes del equipo
- Mensajes de commit en formato Conventional Commits: `feat:`, `fix:`, `refactor:`, `test:`, `docs:`
- Ramas `feature/autenticacion-jwt`, `feature/crud-[entidad]`, mergeadas a `develop`
- Tag `v0.1.0-entrega-1b` en el commit de entrega

2.11 Conclusiones

Parrafo de 150 a 200 palabras por objetivo especifico planteado en la introduccion. Cada conclusion debe: (1) confirmar si el objetivo se logro, (2) mencionar el mayor desafio tecnico encontrado, (3) indicar como se resolvió. Finalizar con una proyeccion concreta de lo que se construira en la Entrega 2 (Semana 12).

2.12 Referencias bibliograficas

Minimo 8 referencias en norma APA 7ma edicion. Incluir:

1. Walls, C. (2022). *Spring Boot in action* (2da ed.). Manning Publications. ISBN 978-1-617-29454-7

2. Spilca, L. (2024). *Spring Security in action* (2da ed.). Manning Publications. ISBN 978-1-633-43797-5
3. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. <https://tools.ietf.org/html/rfc7519>
4. OWASP Foundation. (2021). *OWASP Top 10: 2021*. <https://owasp.org/Top10/>
5. Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, UC Irvine].
6. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
7. Spring Framework Team. (2024). *Spring Security reference documentation*. <https://docs.spring.io/spring-security/reference/>

n aplicaciones web

Criterios de evaluacion

Criterio	%	Nivel alto	Nivel basico
Autenticacion JWT funcional	25	Login, logout y refresh funcionan; token tiene estructura correcta; blacklist Redis operativa	Solo login funciona; sin logout; sin refresh
CRUD con Spring Data JPA	20	CRUD completo con paginacion; Flyway gestiona el esquema; zero concatenacion SQL	CRUD parcial; DDL manual sin Flyway
Seguridad OWASP	15	BCrypt costo 12; cabeceras HTTP; CORS correcto; roles con <code>@PreAuthorize</code>	Solo BCrypt; sin cabeceras ni CORS
Pruebas automatizadas	15	5+ pruebas JUnit en verde; captura <code>mvn test</code>	Menos de 3 pruebas o fallan
Docker Compose	10	<code>docker compose up</code> levanta todos los servicios; health checks; README con instrucciones	Solo backend funciona; sin Redis
Informe tecnico	10	Resumen, marco teorico con 8+ referencias APA, diagramas C4 y secuencia actualizados	Sin marco teorico o referencias insuficientes

Criterio	%	Nivel alto	Nivel basico
Repositorio Git	5	Commits de todos; Conventional Commits; tag de entrega	Solo un integrante con commits
TOTAL	100 %	Convertida a la escala de la asignatura segun normativa UTEQ	

Errores que reducen drasticamente la calificacion

[ANULACION] El sistema no arranca con docker compose up. El docente verifica la aplicacion en tiempo real. Un sistema que no ejecuta equivale a cero en los criterios funcionales (autenticacion, CRUD, Docker), independientemente de la calidad del informe.

[ANULACION] Contraseñas almacenadas en texto plano. Si la columna `password_hash` de la BD contiene texto legible, el criterio de seguridad se anula completamente y se descontara adicionalmente del criterio de autenticacion.

[PENALIZACION GRAVE] Token JWT almacenado en localStorage. `localStorage` es vulnerable a XSS. El `accessToken` debe estar en memoria y el `refreshToken` en una cookie `HttpOnly`. Si el estudiante justifica una alternativa con evidencia, se acepta con reduccion menor.

[PENALIZACION GRAVE] Consultas SQL concatenadas. Cualquier `String sql = "..."` + `variable` dentro del codigo Java es SQL Injection potencial y penaliza el criterio de seguridad y el de CRUD.

Lista de verificacion final

Verificar antes de subir el PDF al SGA	
☐	El informe tiene portada con integrantes, cédulas, URL del repositorio y rama entrega-1b .
☐	El resumen ejecutivo describe los resultados concretos con al menos una métrica numérica.
☐	El marco teórico tiene mínimo 8 referencias APA; al menos 3 son artículos indexados.
☐	El diagrama C4 Nivel 2 está actualizado y muestra backend, frontend, PostgreSQL y Redis.
☐	El diagrama de secuencia del flujo JWT está incluido con los 11 pasos descritos.
☐	El script Flyway <code>V1__schema_inicial.sql</code> es ejecutable en PostgreSQL 16 sin errores.
☐	La tabla de endpoints lista al menos los 10 endpoints de autenticación y CRUD.
☐	La tabla de seguridad documenta al menos 6 controles OWASP con evidencia.
☐	Las capturas de Postman muestran login exitoso con el JWT visible en la respuesta.
☐	Las capturas muestran el CRUD completo: POST, GET lista con paginación, GET ID, PUT, DELETE.
☐	La captura de <code>mvn test</code> muestra al menos 5 pruebas en verde.
☐	La captura de Swagger UI muestra los endpoints documentados.
☐	La tabla de métricas tiene tiempos de respuesta reales medidos con Postman o JMeter.
☐	El <code>docker-compose.yml</code> incluye los 4 servicios con health checks.
☐	El README.md tiene instrucciones de ejecución en 5 pasos ejecutables.
☐	El historial Git muestra commits de todos los integrantes con mensajes Conventional Commits.
☐	El tag <code>v0.1.0-entrega-1b</code> existe en el repositorio y apunta al commit de entrega.
☐	El archivo PDF se llama <code>PFC_Entrega1B_[NombreEquipo].pdf</code> .
☐	El PDF fue adjuntado en la Tarea del SGA antes del cierre de la Semana 9.